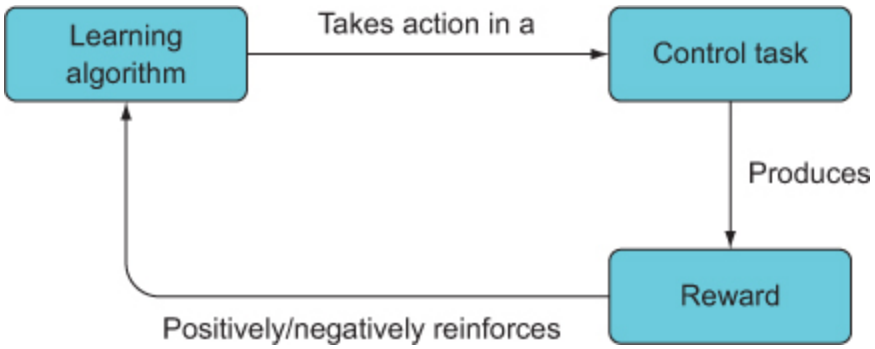


1.3. Dynamic programming versus Monte Carlo

You now know that you can train an algorithm to accomplish some high-level task by assigning the completion of the task a high reward (i.e., positive reinforcement) and negatively reinforce things we don't want it to do. Let's make this concrete. Say the high-level goal is to train a robot vacuum to move from one room in a house to its dock, which is in the kitchen. It has four actions: go left, go right, go forward, and go reverse. At each point in time, the robot needs to decide which of these four actions to take. If it reaches the dock, it gets a reward of +100, and if it hits anything along the way, it gets a negative reward of –10. Let's say the robot has a complete 3D map of the house and has the precise location of the dock, but it still doesn't know exactly what sequence of primitive actions to take to get to the dock.

One approach to solving this is called *dynamic programming* (DP), first articulated by Richard Bellman in 1957. Dynamic programming might better be called *goal decomposition* as it solves complex high-level problems by decomposing them into smaller and smaller subproblems until it gets to a simple subproblem that can be solved without further information.

Rather than the robot trying to come up with a long sequence of primitive actions that will get it to the dock, it can first break the problem down into "stay in this room" versus "exit this room." Since it has a complete map of the house, it knows it needs to exit the room, because the dock is in the kitchen. Yet it still doesn't know what sequence of actions will allow it to